Software Engineering Lab

Lab 39

Bloom Filters

Struktur Data Probabilistik untuk Pengurangan I/O dan Pertahanan terhadap Cache Penetration



Lukman / lukman-ss

Disk I/O Mahal, Cache Penetration Mengancam

Query kunci tidak ada di LSM-Tree memicu ribuan disk read, sementara request non-existent ke cache melintasi backend secara tidak terkendali.

MOTIVASI

Disk I/O adalah operasi paling mahal dalam basis data; setiap read segment menghabiskan sumber daya.
Cache penetration terjadi ketika request untuk kunci tidak existing melewati cache dan membebani backend.
Server backend yang dipanggil berlebihan dapat menyalahi batas kapasitas dan memperburuk latency tail.

BUKTI LAB

Tanpa filter: 10.000 query absent = 100.000 disk reads (10 segment × 10.000 pencarian).
Tanpa filter pada cache: 1.000 penetration attempts = 1.000 backend calls secara linear.
Dengan Bloom filter: disk reads terpangkas menjadi 988 (penurunan drastis 99.01%).

BATASAN SISTEM

Lab memodelkan in-memory store; di sistem produksi nyata latensi round-trip jaringan memperparah beban.

Di lingkungan production dengan ratusan ribu record, setiap pencarian kunci tidak ada di sebuah basis data berbasis Log-Structured Merge Tree (LSM-Tree) memaksa pembacaan berulang ke semua segment SSTable yang ada di disk. Akibatnya, ribuan operasi I/O yang sia-sia tetap dieksekusi hanya untuk mengonfirmasi bahwa data yang dicari memang tidak ada. Kondisi ini diperparah oleh fenomena cache penetration, di mana banjir permintaan terhadap identifier acak atau non-existent melompati layer in-memory cache dan menghantam downstream database secara langsung, memicu kelelahan connection pool dan lonjakan latensi p99.

Probabilistic filters prevent unnecessary backend dives. One false positive is better than a thousand disk reads.

Bloom Filter: Definisi dan Jaminan Asimetris

Struktur probabilistik yang memberikan dua jaminan berbeda: zero false negatives dan configurable false positives.

KOMPONEN TEORI

Bit array berukuran m bit yang awalnya seluruhnya diinisialisasi dengan nilai 0.
k fungsi hash seragam yang memetakan elemen kunci ke indeks posisi bit array.
False negative rate 0% mutlak: elemen yang ada tidak akan pernah dilaporkan absen.
False positive rate ε: probabilitas elemen absen dilaporkan hadir dapat dikonfigurasi.

MODEL MENTAL

Definitely Not in Set: jika pengecekan menghasilkan false, kunci pasti tidak ada dalam koleksi.
Possibly in Set: jika pengecekan bernilai true, kunci kemungkinan ada atau terjadi collision.
Asymmetric guarantee menjadikannya gerbang penjaga ideal untuk operasi I/O yang berbiaya mahal.

BATASAN STRUKTUR

Bloom filter standar bersifat statis dan secara fundamental tidak mendukung operasi penghapusan.

Bloom filter adalah struktur data probabilistik yang didesain secara khusus untuk menjawab pertanyaan keanggotaan himpunan dengan efisiensi memori yang sangat tinggi. Karakteristik utama dari struktur ini terletak pada jaminan asimetrisnya yang sangat menguntungkan arsitektur sistem. Ketika fungsi pemeriksaan mengembalikan nilai false, sistem memiliki kepastian seratus persen bahwa elemen tersebut tidak pernah dimasukkan sebelumnya. Sebaliknya, ketika fungsi mengembalikan nilai true, terdapat kemungkinan kecil bahwa itu merupakan false positive akibat tumpang tindih bit hash dari elemen-elemen lain.

Zero false negatives is a feature, not a bug. Trade certainty for compression.

Fondasi Matematika Ukuran Bit Array dan Hash Optimal

Ukuran alokasi memori m dan jumlah fungsi hash k diturunkan secara presisi dari kapasitas elemen n dan target error rate ε.

FORMULA TERTUTUP

m = ceil(-n × ln(ε) / (ln 2)²) → total alokasi bit array yang dibutuhkan
k = round((m/n) × ln 2) → kuantitas probe fungsi hash paling optimal
Bits per element konstan: m/n ≈ -1.44 × $\log_2(\varepsilon)$ bit per entitas data

CONTOH PERHITUNGAN

Untuk n = 10.000 elemen dengan target false positive rate ε = 0.01 (1%):
m = ceil(-10000 × ln(0.01) / 0.48045) = 95.851 bit (hanya sekitar 11.7 Kilobyte memori).
k = round((95851/10000) × 0.69315) = 7 fungsi hash independen.
Rasio memori teoritis: 9.59 bit per elemen untuk jaminan 99% filtrasi tepat sasaran.

Target FP Rate ε	Bits per Elemen	Jumlah Hash k
0.10 (10%)	4.79 bits	3 hash probes
0.01 (1%)	9.59 bits	7 hash probes
0.001 (0.1%)	14.38 bits	10 hash probes

Penentuan dimensi bit array dan jumlah fungsi hash bukanlah tebakan empiris melainkan kalkulasi matematis yang ketat. Menggunakan persamaan klasik yang dianalisis ulang oleh Kirsch dan Mitzenmacher, kapasitas bit m dihitung untuk meminimalkan probabilitas kolusi hash pada tingkat saturasi bit array tertentu. Pada target false positive rate satu persen, setiap elemen hanya membutuhkan alokasi memori sebesar sembilan koma lima puluh sembilan bit. Dibandingkan dengan menyimpan representasi string mentah yang memakan puluhan byte, Bloom filter menghadirkan kompresi ruang penyimpanan puluhan kali lipat.

Math gives you compression; testing gives you truth.

Optimasi Double Hashing Kirsch-Mitzenmacher

Mereduksi kompleksitas komputasi k fungsi hash independen menjadi hanya dua hash dasar via kombinasi linier.

FORMULA LINIER

h1 = FNV-1a 64-bit hash dari representasi byte kunci
h2 = h1 XOR (h1 rotasi 17 bit | h1 rotasi 47 bit) sebagai hash ortogonal kedua
Posisi bit ke-i: pos_i = (h1 + i × h2) mod m untuk indeks probe i = 0 hingga k-1

KEUNTUNGAN ALGORITMIK

Menghilangkan kebutuhan mengeksekusi k fungsi hash kriptografis yang berulang dan berat.
Secara matematis terbukti menghasilkan distribusi bit seragam setara dengan k hash acak terpisah.
Zero memory allocations: seluruh kalkulasi probe berjalan murni pada register CPU.

Salah satu tantangan implementasi Bloom filter berkinerja tinggi adalah mahalnya komputasi jika sistem harus mengeksekusi tujuh hingga sepuluh algoritma hash yang berbeda untuk setiap kunci. Teknik double hashing yang diperkenalkan oleh Kirsch dan Mitzenmacher pada tahun 2006 membuktikan bahwa dua nilai hash dasar 64-bit cukup untuk mensimulasikan k fungsi hash independen tanpa degradasi akurasi false positive. Di Lab 39, hash pertama dihitung menggunakan FNV-1a, dan hash kedua dihasilkan melalui rotasi xorshift pada register 64-bit.

```
func baseHashes(key []byte) (uint64, uint64) {
    h := fnv.New64a()
    _, _ = h.Write(key)
    h1 := h.Sum64()
    h2 := h1 ^ (h1>>17 | h1<<47)
    if h2 == 0 { h2 = 0xdeadbeefdeadbeef }
    return h1, h2
}
```

Linear probing with two hashes achieves the same distribution as k independent hashes.

internal/bloom/bloom.go:65-83

Arsitektur Bit Array Word-Aligned dan API Inti

Representasi memori berbasis slice uint64 yang efisien dengan antarmuka Add dan Check yang padat.

```
type Filter struct {
    bits []uint64 // array bit 64-bit word-aligned
    m    uint     // total kapasitas bit
    k    uint     // jumlah probe per elemen
}
```

OPERASI BITWISE CEPAT

Menandai bit: f.bits[pos/64] |= 1 << (pos % 64)
Memeriksa bit: if f.bits[pos/64] & (1 << (pos % 64)) == 0 { return false }
Ukuran kata dialokasikan dengan padding kelipatan 64 bit: words := (m + 63) / 64

MUTASI DAN AKSES

Metode Add menandai tepat k bit dalam bit array untuk setiap kunci.
Metode Check melakukan short-circuiting: langsung kembali false saat menemukan bit 0 pertama.

Untuk memaksimalkan performa pada arsitektur perangkat keras modern, bit array tidak diimplementasikan sebagai kumpulan boolean individual, melainkan dikemas ke dalam slice uint64 64-bit yang word-aligned. Operasi penentuan posisi bit memanfaatkan pembagian integer cepat untuk indeks slice dan operasi modulo bitwise untuk pergeseran bit (bit shifting). Pendekatan ini membuat operasi Add dan Check berjalan dalam orde nanodetik serta ramah terhadap CPU cache line.

Bit manipulation is the soul of Bloom filters. Everything else is interface.

Arsitektur Penyimpanan LSM-Tree Berbasis Segment Filter

Integrasi Bloom filter pada setiap immutable SSTable segment untuk eliminasi I/O disk.

DESAIN KOMPONEN

Segment: representasi immutable komponen SSTable berisi key-value map dan instance Bloom filter.
LSMStore: agregator multi-segment yang mengoordinasikan pencarian data dari segment termuda ke terlama.
Instrumentasi diskReads: pencatatan atomik setiap kali isi segment diakses secara fisik.

```
func (s *LSMStore) Get(key string) (string, bool) {
    for i := len(s.segments) - 1; i >= 0; i-- {
        seg := s.segments[i]
        if seg.filter != nil && !seg.filter.Check(keyBytes) {
            continue // disk read diabaikan sepenuhnya!
        }
        atomic.AddUint64(&s.diskReads, 1)
        // load data block...
    }
}
```

DAMPAK PADA PIPELINE

Pencarian kunci yang tidak ada langsung dieliminasi sebelum memicu operasi

Penerapan paling terkenal dari Bloom filter di dunia industri adalah mesin penyimpanan Log-Structured Merge Tree (LSM-Tree) seperti RocksDB, Cassandra, dan Bigtable. Karena data pada LSM-Tree tersebar di banyak file segment SSTable yang immutable akibat operasi flush periodik, pencarian record yang tidak ada mengharuskan sistem membaca setiap segment satu per satu. Dengan menyematkan Bloom filter kecil di header setiap segment, sistem dapat memangkas pencarian disk hampir sepenuhnya.

Prune before you peek. Bloom filters turn O(N) disk scans into O(1) rejections.

Pengurangan 99% Disk I/O pada Pencarian Kunci Absen

Pengujian 10.000 query kunci absen pada 10 segment memangkas pembacaan dari 100.000 menjadi 988 kali.

Parameter	Tanpa Filter	Dengan Bloom Filter
Jumlah Segment Data	10 segment	10 segment
Total Kunci Tersimpan	20.000 record	20.000 record
Kueri Kunci Absen	10.000 query	10.000 query
Total Pembacaan Segment	100.000 reads	988 reads
Efisiensi Pengurangan I/O	Baseline (0%)	99.01% penurunan

Eksperimen eksekusi pada skenario kedua mendemonstrasikan secara kuantitatif besarnya penghematan sumber daya komputasi. Ketika sepuluh ribu kueri untuk data yang tidak ada dikirimkan ke LSMStore dengan sepuluh segment data, arsitektur tanpa filter membukukan tepat seratus ribu pembacaan disk. Saat Bloom filter diaktifkan dengan konfigurasi error satu persen, angka tersebut terpangkas drastis menjadi hanya 988 pembacaan. Pengurangan sebesar 99.01% ini secara langsung melipatgandakan daya tampung query engine.

ANALISIS AKUMULASI DISK

Pada sistem tanpa filter, setiap kueri absen memicu pembacaan 10 segment sekaligus secara berurutan.
Dengan Bloom filter ($\varepsilon = 0.01$), hanya 988 operasi pembacaan yang bocor akibat batas toleransi false positive.

KORELASI TEORI

Tingkat kebocoran I/O sebesar 0.988% konsisten sempurna dengan target false positive rate 1%.

>99% of absent queries are filtered before touching storage.

Anatomi Serangan Cache Penetration pada Backend

Permintaan berbahaya untuk data non-existent melewati lapisan cache dan membebani basis data primer.

MEKANISME KERENTANAN

Klien mengirimkan request dengan identifier yang sengaja dibuat acak atau tidak valid.
In-memory cache memeriksa tabel hash lokal, mencatat cache miss, dan meneruskan query ke backend.
Backend database mengeksekusi query lambat di disk hanya untuk mengembalikan status null/empty.
Karena data null umumnya tidak disimpan di cache, kueri berulang terus menembus ke backend.

SKENARIO SERANGAN LAB

Basis data hilir hanya memuat tiga entitas produk yang valid (product:1, 2, 3).
Sistem diserbu 5.000 kueri penetrasi simultan dengan pola non-existent-product-{i}.
Cache standar tanpa pengamanan mencatat tepat 5.000 pemanggilan backend langsung (100% tembus).

RISIKO PRODUKSI

Peningkatan konsumsi CPU database, kehabisan koneksi pool, dan potensi denial of service (DoS).

Cache penetration adalah skenario kegagalan arsitektur caching yang sangat umum namun kerap diabaikan. Berbeda dari cache breakdown atau cache avalanche yang melibatkan kunci valid yang kadaluwarsa secara bersamaan, penetration terjadi ketika kunci yang diminta memang tidak pernah ada di dalam database. Arsitektur caching konvensional tidak memiliki mekanisme memori untuk mengingat ketiadaan data, sehingga setiap query acak akan meluncur bebas menuju basis data utama.

Defend the cache before it defends you. Block absent keys at the gate.

Bloom Filter sebagai Admission Gate pada Arsitektur Cache

Pemeriksaan keanggotaan di tepi gerbang cache menolak akses ke backend database untuk kunci asing.

```
func (c *Cache) Get(key string) (string, bool) {
    if c.filter != nil && !c.filter.Check([]byte(key)) {
        return "", false // DITOLAK SEBELUM MENYENTUH BACKEND
    }
    // Cek memori lokal cache...
    // Jika miss, baru panggil c.backend.Fetch(key)
}
```

Dengan memasang Bloom filter thread-safe (SyncFilter) sebagai admission gate di bagian depan cache layer, kita menciptakan dinding pertahanan yang sangat kokoh. Setiap kali request tiba, sistem terlebih dahulu menanyakan keberadaan kunci pada filter. Jika filter menyatakan kunci pasti tidak ada, request langsung dikembalikan sebagai respons negatif tanpa pernah mengalokasikan thread database. Backend hanya akan dihubungi jika kunci tersebut lolos pemeriksaan awal.

KARAKTERISTIK OPERASIONAL

Kunci yang valid diisi ke dalam filter saat inisialisasi atau saat pertama kali divalidasi dari backend.
Permintaan untuk kunci asing langsung ditolak dalam 150 nanodetik via kalkulasi bitwise di memori RAM.
Database hilir terlindungi sepenuhnya dari lonjakan lalu lintas query non-existent.

HASIL VERIFIKASI

Dari 5.000 percobaan penetrasi kunci asing, tepat 0 kueri yang berhasil menembus ke backend.

A Bloom filter at the cache edge stops 100% of absent-key attacks.

Pembuktian Invarian Nol False Negative pada 10.000 Kunci

Pengujian unit test memverifikasi seluruh kunci yang dimasukkan selalu terdeteksi hadir tanpa pengecualian.

```
func TestNoFalseNegatives(t *testing.T) {
    bf := New(10000, 0.01)
    for i := 0; i < 10000; i++ {
        bf.Add([]byte(keys[i]))
    }
    for _, k := range keys {
        if !bf.Check([]byte(k)) {
            t.Fatalf("False negative detected for key: %s", k)
        }
    }
}
```

Invarian paling fundamental yang wajib dibuktikan pada Bloom filter adalah ketiadaan false negative. Jika sebuah filter secara keliru menyatakan suatu kunci tidak ada padahal kunci tersebut sebenarnya tersimpan di basis data, maka sistem akan mengalami kehilangan data fungsional (silent data loss). Pengujian TestNoFalseNegatives secara teliti memverifikasi sepuluh ribu kunci unik yang dimasukkan dan membuktikan bahwa seluruhnya berhasil ditemukan kembali tanpa ada satu pun kegagalan.

HASIL PENGUJIAN OTOMATIS

Total kunci yang dimasukkan: 10.000 string unik
Pengecekan keanggotaan ulang: 10.000 kali pengujian
False negative yang ditemukan: Tepat 0 kasus (0.00% error rate)
Status verifikasi Go test: PASS (durasi eksekusi 0.00 detik)

KESEPAKATAN ARSITEKTUR

Sistem aman mengabaikan pembacaan disk karena tidak ada risiko kehilangan data yang valid.

False negative = 0% is the only guarantee you need for defensive filtering.

internal/bloom/bloom_test.go:12–28

Validasi Tingkat False Positive Empiris pada 50.000 Kueri

Pengukuran statistik mencatat false positive sebesar 0.996%, sangat presisi terhadap target desain 1.00%.

Metrik Uji	Target Teori	Hasil Terukur
Kapasitas Filter n	10.000 elemen	10.000 elemen terisi
Target False Positive ε	0.0100 (1.00%)	0.0098 - 0.00996 (~0.99%)
Kueri Kunci Asing	50.000 kueri	50.000 kueri unik
Deteksi False Positive	~500 kueri	489 - 498 deteksi
Toleransi Kebisingan	2.5 × target (2.5%)	Lolos verifikasi

Karakteristik kedua dari Bloom filter adalah laju false positive yang dapat dikontrol. Pada unit test TestFalsePositiveRate, filter yang telah memuat sepuluh ribu kunci diuji dengan lima puluh ribu kueri kunci acak yang dipastikan tidak pernah dimasukkan. Hasil eksekusi mencatat 489 hingga 498 deteksi false positive, yang merepresentasikan tingkat error sebesar 0.98% hingga 0.996%. Angka ini bersesuaian sempurna dengan batas teoretis satu persen yang ditetapkan pada parameter konstruktor.

KONSISTENSI FORMULA

Persamaan FP rate: ε ≈ (1 - e^(-k×n/m))^k
Dengan m/n = 9.585 dan k = 7, probabilitas teoretis menghasilkan 0.00997. Hasil empiris 498 dari 50.000 kueri membuktikan keandalan formula Kirsch-Mitzenmacher.

STABILITAS RUNNING

Pengujian berulang pada berbagai seed menghasilkan deviasi standar di bawah 0.03%.

Empirical FP converges to theoretical within noise margins.

internal/bloom/bloom_test.go:30–55

Verifikasi Presisi Dimensi Memori dan Jumlah Hash

TestOptimalSizing membuktikan perhitungan m dan k di kode Go identik dengan turunan matematis tertutup.

```
func TestOptimalSizing(t *testing.T) {
    const n = 1000
    const eps = 0.01
    bf := New(n, eps)
    expectedM := uint(math.Ceil(-float64(n)*math.Log(eps)/(math.Log(2)*math.Log(2))))
    expectedK := uint(math.Round(float64(expectedM)/float64(n)*math.Log(2)))
    // Assert bf.M() == expectedM && bf.K() == expectedK
}
```

HASIL VERIFIKASI NUMERIK

Untuk n = 1.000 dan ε = 0.01: expectedM = 9.586 bit, aktual bf.M() = 9.586 bit (cocok presisi).
Jumlah fungsi hash: expectedK = 7 probes, aktual bf.K() = 7 probes.
Rasio efisiensi: 9.586 bit per elemen (memenuhi batasan ketat 9.5 <= bits <= 10.5).

KEPASTIAN ALOKASI

Memastikan tidak ada alokasi bit berlebih yang membuang memori atau alokasi kurang yang merusak akurasi.

Implementasi yang andal tidak boleh memiliki penyimpangan pembulatan aritmatika float yang dapat mengubah dimensi alokasi. TestOptimalSizing mengunci kebenaran implementasi fungsi optimalM dan optimalK terhadap rumus formalnya. Dengan mengevaluasi alokasi bit dan probe hash pada skala uji, tes ini memastikan bahwa memori dialokasikan secara optimal tanpa pemborosan satu byte pun seraya mempertahankan sifat matematis kurva error.

Formula is law. Implementation is faithful.

internal/bloom/bloom_test.go:57–79

Keamanan Akses Konkuren dengan Race Detector Bersih

Pengujian multi-goroutine dengan 8 writer dan 16 reader simultan lolos pengujian tanpa data race.

DESAIN BEBAN KONKUREN

8 goroutine writer secara bersamaan menambahkan 1.000 elemen unik ke SyncFilter.
16 goroutine reader mengeksekusi 8.000 pengecekan Check secara paralel.
Pengujian dijalankan dengan flag kompetitif Go race detector: go test -race.

```
type SyncFilter struct {
    mu sync.RWMutex
    f  *Filter
}
Add memegang Lock eksklusif, sementara Check memegang RLock bersama.
Throughput pembacaan tetap maksimal di bawah beban kerja read-heavy.
```

HASIL EKSEKUSI

Zero data race, zero deadlock, dan integritas bit array tetap terjaga sepenuhnya.

Dalam arsitektur backend berkinerja tinggi, struktur filter akan diakses oleh ratusan worker goroutine secara paralel. Tipe Filter dasar sengaja dirancang tanpa sinkronisasi untuk kecepatan maksimal pada skenario immutable seperti segment SSTable. Untuk skenario dinamis seperti admission cache, paket menyediakan tipe pembungkus SyncFilter yang mengamankan mutasi menggunakan sync.RWMutex. Pengujian membuktikan bahwa operasi read-heavy dapat berjalan serempak tanpa memicu race condition.

RWMutex makes Bloom filters concurrent-ready without complexity.

internal/bloom/bloom_test.go:88–117

Pengujian Mitigasi Cache Penetration pada Beban Serangan

Simulasi 1.000 kueri penetrasi membuktikan Bloom filter memblokir seluruh akses ke backend tanpa kebocoran.

SKENARIO INTEGRASI SISTEM

Backend mock menyimpan tiga data pengguna valid (user:101, user:102, user:103).
Cache tanpa proteksi diekspos terhadap 1.000 kueri acak (hacker-query-{i}).
Hasil unprotected cache: mencatat tepat 1.000 backend calls (100% tembus).

EVALUASI PROTECTED CACHE

Cache terproteksi diinisialisasi dengan admission gate SyncFilter (n = 1000, ε = 0.01).
Kueri data sah (user:101) dilayani langsung dari memori dengan 0 panggilan backend.
Serangan 1.000 kueri asing ditahan di gerbang depan, menghasilkan 0 panggilan backend.

AMBANG TOLERANSI TEST

Ambang batas kelulusan pengujian: penetrasi backend <= 30 kueri (toleransi false positive).
Aktual terukur: 0 penetrasi tercatat (tingkat filtrasi absolut 100%).

Pengujian integrasi TestCachePenetrationMitigation memvalidasi efektivitas arsitektur admission gate dalam meredam serangan penetrasi kueri acak. Ketika seribu permintaan berbahaya dikirimkan secara berurutan, cache standar tanpa filter mengalami kebobolan total dengan meneruskan setiap permintaan ke backend database. Sebaliknya, cache yang dipersenjatai dengan Bloom filter sukses mengidentifikasi bahwa kunci-kunci tersebut tidak pernah terdaftar, menolak seluruhnya sebelum menyentuh sumber data utama.

Block penetration at the gate, not at the database.

Tabel Komparasi Performa Komprehensif Arsitektur Lab

Rekapitulasi kuantitatif dampak penerapan Bloom filter pada throughput I/O dan keamanan backend.

Skenario Evaluasi	Tanpa Bloom Filter	Dengan Bloom Filter	Efisiensi Peningkatan
LSM-Tree Disk Reads (10k query)	100.000 pembacaan	988 pembacaan	99.01% penurunan I/O
Cache Penetration Hits (5k query)	5.000 backend hits	0 backend hits	100% perlindungan
Latensi Filter Probe	0 ns (tanpa gate)	~150 ns per lookup	Akses memori sub-mikrodetik
Overhead Memori (10k elemen)	0 byte	11.7 Kilobyte	Sangat ringkas (9.59 bit/item)

Tabel komparasi di atas mengkristalisasi seluruh temuan eksperimental Lab 39. Perbedaan performa antara sistem dengan dan tanpa Bloom filter berada pada skala yang sangat kontras. Dengan mengorbankan memori sebesar 11.7 Kilobyte dan waktu komputasi hash sekitar 150 nanodetik, sistem mampu mengeliminasi lebih dari 99% pembacaan disk yang memakan waktu milidetik serta menghindarkan basis data hilir dari kelumpuhan akibat serangan cache penetration.

ANALISIS RETURN ON INVESTMENT

Dengan biaya memori hanya 11.7 KB, sistem menghemat puluhan ribu operasi I/O disk yang lambat.
Trade-off latensi 150 nanodetik di memori mengeliminasi latensi disk read sebesar 5 hingga 10 milidetik.

KESIMPULAN METRIK

Numbers don't lie: one filter saves a mountain of I/O.

cmd/demo/main.go:59-128

Verifikasi Perilaku Filter Kosong: Penolakan Aman Awal

Memastikan filter yang baru diinisialisasi menolak seluruh kueri secara konservatif tanpa false hit.

STATE BIT AWAL

Konstruktor New mengalokasikan slice uint64 dengan seluruh kata bernilai 0.
Setiap pemanggilan Check pada filter kosong akan mendeteksi bit 0 pada probe pertama.
Short-circuit evaluation langsung mengembalikan nilai false seketika.

```
func TestEmptyFilter(t *testing.T) {
    bf := New(100, 0.01)
    if bf.Check([]byte("any-key")) {
        t.Errorf("Empty filter should not return true for arbitrary key")
    }
}
```

KEAMANAN FAIL-SAFE

Filter kosong bertindak sebagai gerbang tertutup yang aman, mencegah false positive dini sebelum data dimasukkan.

Pengujian kondisi batas (boundary testing) sangat penting untuk menjamin kestabilan perangkat lunak. Pada TestEmptyFilter, sistem memastikan bahwa filter yang baru dibuat tidak mengandung bit sampah yang tertinggal di memori. Sebelum ada kunci yang dimasukkan via Add, setiap kueri harus selalu menghasilkan respons false. Perilaku ini memastikan bahwa dalam sistem penyimpanan, segment SSTable yang masih kosong tidak akan pernah memicu pembacaan disk palsu.

Empty filter is a safe filter. Start conservative, add keys, relax guarantees.

Analisis Trade-off Memori versus False Positive Rate

Sensitivitas pemilihan nilai epsilon terhadap konsumsi ruang memori dan jumlah fungsi hash yang dibutuhkan.

Target False Positive ε	Bits/Elemen	Hash Probes k	Memori untuk 1 Juta Elemen	Karakteristik Beban
0.10 (10% error)	4.79 bits	3 probes	599 Kilobyte	Pruning segment disk kasar
0.01 (1% error)	9.59 bits	7 probes	1.17 Megabyte	Standar industri umum (RocksDB)
0.001 (0.1% error)	14.38 bits	10 probes	1.75 Megabyte	Proteksi cache penetration ketat
0.0001 (0.01% error)	19.17 bits	13 probes	2.34 Megabyte	Biaya komputasi hash tinggi

Merancang arsitektur sistem selalu melibatkan kompromi trade-off. Memilih target false positive rate yang tepat memerlukan pemahaman mendalam atas biaya relatif antara alokasi RAM dan dampak I/O. Mengonfigurasi filter dengan epsilon 1% membutuhkan sekitar 9.6 bit per elemen, yang merupakan titik seimbang paling populer di industri. Menurunkan error hingga 0.01% melipatgandakan kebutuhan memori dan probe hash, yang berpotensi membebani CPU cache tanpa memberikan keuntungan I/O yang sebanding.

PRINSIP ARSITEKTUR MEMORI

Mereduksi false positive dari 1% ke 0.1% membutuhkan peningkatan memori sekitar 50% dan 3 probe hash tambahan.
Di sebagian besar sistem database, nilai ε = 0.01 (1%) merupakan sweet spot efisiensi optimal.

Memory is cheap, latency is expensive. Spend on compression wisely.

internal/bloom/bloom.go:85–91

Menargetkan error mendekati nol membutuhkan memori eksponensial yang

Karakteristik Fungsi Hash: Efisiensi FNV-1a versus Keamanan

Evaluasi batas ketahanan FNV-1a terhadap manipulasi kunci adversarial dan mitigasi pada skala produksi.

KELEBIHAN FNV-1A DI LAB

Eksekusi sangat cepat dengan overhead instruksi CPU yang minimal pada operasi per-byte.
Distribusi bit seragam untuk kunci terdistribusi acak alami (seperti UUID, sequence ID, slug).
Murni menggunakan Go standard library (hash/fnv) tanpa dependensi eksternal.

VEKTOR KERENTANAN ADVERSARIAL

FNV-1a bukan fungsi hash kriptografis yang tahan benturan (collision-resistant).
Penyerang luar dapat merekayasa sekumpulan string kunci khusus yang sengaja menghasilkan hash yang sama.
Serangan hash flooding dapat mendegradasi false positive rate sistem secara artifisial.

REKOMENDASI PRODUKSI ADVERSARIAL

Gunakan SipHash dengan kunci rahasia acak per instance untuk endpoint publik yang tidak terpercaya.
Untuk sistem internal tertutup (seperti SSTable segment private), FNV-1a atau

Pilihan fungsi hash menentukan kecepatan sekaligus profil keamanan Bloom filter. Lab 39 mengadopsi FNV-1a dari Go standard library karena efisiensinya yang tinggi dan nol dependensi eksternal. Untuk lingkungan data internal tertutup seperti segment SSTable, FNV-1a bekerja dengan sangat baik. Namun, jika filter terekspos langsung ke input publik dari pengguna yang tidak terpercaya, engineer wajib mempertimbangkan hash berkunci seperti SipHash guna mencegah serangan hash collision terencana.

Speed today, security tomorrow. Choose hash by threat model.

Batasan Fundamental: Ketiadaan Dukungan Operasi Delete

Mengapa bit array tidak boleh dihapus dan bagaimana arsitektur modern mengatasi keterbatasan ini.

AKAR MASALAH PENGHAPUSAN

Bit dalam array bersifat tumpang tindih (shared): satu bit dapat disetel oleh puluhan kunci berbeda.
Mereset bit dari 1 ke 0 saat 'menghapus' kunci A akan merusak jejak hash kunci B dan C.
Dampaknya fatal: merusak invarian zero false negative sehingga data yang ada dilaporkan hilang.

SOLUSI ARSITEKTURAL DI LAB

Imutabilitas Segment: SSTable segment bersifat append-only dan tidak pernah dimutasi setelah ditulis.
Rebuild saat Compaction: filter baru dibangun dari nol ketika beberapa segment digabungkan.
Cache Gate: filter hanya melakukan Add saat verifikasi data; kadaluwarsa ditangani via rebuild periodik.

STRUKTUR ALTERNATIF

Gunakan Counting Bloom Filter (alokasi memori 3–4× lipat) atau Cuckoo Filter jika mutasi delete wajib didukung.

Kelemahan paling menonjol dari Bloom filter standar adalah ketidakmampuannya mendukung operasi delete. Karena beberapa kunci dapat mengaktifkan bit yang sama persis, membalikkan nilai bit menjadi 0 akan serta-merta merusak status keanggotaan kunci lain yang sah, memicu false negative yang mematikan integritas data. Di Lab 39, batasan ini diselaraskan dengan pola imutabilitas segment LSM-Tree: filter hanya dibuat sekali bersama segment dan dibuang sepenuhnya saat segment di-merge.

Deletion requires counting. Immutability saves correctness.

Pertimbangan Operasional untuk Deployment di Lingkungan Produksi

Strategi mitigasi saturasi kapasitas, serialisasi persistensi, dan pemantauan metrik false positive.

RISIKO SATURASI KAPASITAS

False positive rate terikat ketat pada kapasitas n awal yang diestimasikan. Jika jumlah elemen yang dimasukkan melebihi n, kerapatan bit array mendekati 1 dan filter menjadi tidak berguna.
Mitigasi: berikan alokasi margin kapasitas 20–30% atau gunakan Scalable Bloom Filter dinamis.

SERIALISASI DAN DETERMINISME

Filter yang disimpan ke file SSTable disk harus menyertakan metadata m, k, dan representasi byte slice uint64.
Pastikan seed fungsi hash selalu konstan antar proses agar rekonstruksi pembacaan konsisten.

MONITORING PRODUKSI

Ukur metrik empiris rasio query yang ditolak filter versus query yang tembus ke penyimpanan.
Pasang alert jika efisiensi filtrasi menurun drastis, mengindikasikan saturasi kapasitas elemen.

Menerapkan Bloom filter di lingkungan produksi membutuhkan disiplin operasional yang matang. Salah satu jebakan terbesar adalah meremehkan jumlah elemen data di masa depan. Begitu jumlah elemen aktual melampaui kapasitas n yang dirancang, bit array akan mengalami saturasi di mana hampir semua bit bernilai 1, menyebabkan false positive rate meroket hingga 100%. Selain itu, serialisasi biner ke storage membutuhkan determinisme seed hash yang ketat di seluruh instance service.

Production is about defaults and failure modes, not benchmarks.

internal/bloom/bloom_test.go:88–117

Rekapitulasi Verifikasi Suite Pengujian dan Tolok Ukur

Seluruh unit test, pengujian integrasi, benchmark, dan race detector terkonfirmasi lolos sempurna.

TEST SUITE BLOOM PACKAGE

TestNoFalseNegatives: PASS (0 false negative pada 10.000 kunci teruji).
TestFalsePositiveRate: PASS (terukur 0.98% - 0.996% vs target 1.00%).
TestOptimalSizing: PASS (m = 9.586 bit, k = 7 probes terverifikasi presisi).
TestEmptyFilter: PASS (penolakan aman pada kondisi awal terbukti).
TestSyncFilterConcurrency: PASS (bersih tanpa data race pada 24 worker konkuren).

TEST SUITE STORE PACKAGE

TestLSMStoreWithAndWithoutFilter: PASS (99.01% eliminasi pembacaan segment disk).
TestCachePenetrationMitigation: PASS (100% kueri penetrasi tertahan di admission gate).

BENCHMARK PERFORMANCE

BenchmarkFilterAdd: ~500 ns per operasi penambahan.
BenchmarkFilterCheck: ~150 ns per operasi evaluasi probe bitwise.

Kualitas implementasi rekayasa perangkat lunak diukur dari kekokohan pengujian yang mendasarinya. Seluruh klaim teoritis dan arsitektural di Lab 39 didukung oleh rangkaian pengujian otomatis yang komprehensif. Mulai dari kebenaran invarian dasar, kepatuhan batas error empiris, skalabilitas konkurensi dengan Go race detector, hingga simulasi integrasi tingkat sistem, seluruhnya terverifikasi lulus tanpa catatan cacat implementasi.

Test coverage is the guardrail against production regret.

Panduan Reproduksi Mandiri dan Verifikasi Eksekusi Lab

Langkah-langkah mereproduksi hasil pengujian dan eksekusi demo interaktif menggunakan Go murni.

```
cd labs/39-bloom-filters
go test -v ./...                # Eksekusi seluruh rangkaian unit test
go test -race ./...             # Verifikasi integritas konkurensi thread
go run ./cmd/demo               # Jalankan simulasi tiga skenario interaktif
```

STRUKTUR REPOSITORI

internal/bloom/bloom.go: Implementasi inti Filter dan SyncFilter thread-safe.
internal/store/lsm.go: Model LSM-Tree store dengan pruning segment disk.
internal/store/cache.go: In-memory cache dengan Bloom admission gate.
cmd/demo/main.go: Program CLI demonstrasi ketiga skenario lab.

LINGKUNGAN DEPENDENSI

Go 1.22+ standard library murni tanpa ketergantungan paket pihak ketiga.

Reprodusibilitas adalah prinsip utama dari seri Software Engineering Lab. Seluruh kode sumber di Lab 39 dirancang murni menggunakan Go standard library sehingga dapat langsung dikloning dan dijalankan di sistem operasi mana pun tanpa instalasi dependensi rumit. Perintah go test dan go run ./cmd/demo memungkinkan setiap software engineer untuk secara mandiri memverifikasi seluruh angka metrik dan perilaku sistem yang dipaparkan dalam dokumen ini.

Reproduce or reject. Standard library enables instant verification.

Eksperimen kecil, interpretasi yang teliti.

Probabilistic structures give you certainty through approximation.

Software Engineering Lab 39 – Bloom Filters

internal/bloom/bloom.go • internal/bloom/bloom_test.go
internal/store/lsm.go • internal/store/cache.go
cmd/demo/main.go • README.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab`

`labs/39-bloom-filters`

Lukman / lukman-ss

Sumber snapshot commit: c977525a
Seluruh test suite terverifikasi PASS (Race-Clean Go 1.22).
Invarian terbukti: zero false negatives, FP rate ~1%, reduksi I/O >99%.